

OCC: SOFTWARE ENGINEER YEAR 2

SAQA ID

NQF 6

Software design and testing C#

SDT621

SUMMATIVE ASSESSMENT

2026



Table of Contents

DECLARATION OF AUTHENTICITY	3
QUALIFICATION AND ASSESSMENT DETAILS	4
Instructions:	5
Submission Requirements	5
Evidence Collection Requirements	5
To support your assessment submission, you must:	5
GitHub Submission Requirement	5
Summative Assessment Criteria:	6
Example of Collecting Evidence for your submission pdf:	6
SECTION A — Knowledge Questions (40 Marks)	7
True / False Questions [10]	14
SECTION B — Short Practical Questions (10 Marks)	15
Question 1	15
SECTION C: Scenario-Based Practical [50 Marks]	16
Question 1	16
SECTION C.2 — Button Event Logic Implementation (20 Marks)	16
SECTION C.3 — Software Testing Report (15 Marks)	17
SECTION C.4 — GitHub Version Control Evidence (5 Marks)	17
SECTION A — Knowledge Questions (40 Marks)	18
SECTION B — Short Practical Questions (10 Marks)	19
SECTION C — Scenario-Based Practical (50 Marks)	20
SECTION C.2 — Button Event Logic Implementation (20 Marks)	20
SECTION C.3 — Software Testing Report (15 Marks)	20
SECTION C.4 — GitHub Version Control Evidence (5 Marks)	21
SECTION C Mapping Summary	21

DECLARATION OF AUTHENTICITY

I (FULL NAME) | Tiaan Gerber |

hereby declare that the contents of this assessment | SUMMATIVE ASSESSMENT | Software
design and testing C# |

are entirely my own work, completed by me without any paraphrasing/copying, or presented as my own work not accessed from any **AI Apps, example ChatGPT, Co-pilot, Perplexity, or any other App.**

I have read and understood the assessment requirements and the Examination Rules and Regulations.

Signature:

Date: 5/27/2026

QUALIFICATION AND ASSESSMENT DETAILS

Programme Title	OCC: SOFTWARE ENGINEER
Module Title	Software design and testing C#
Module Code	SDT621
NQF Level	6
Assessment Type	SUMMATIVE ASSESSMENT
Hand Out Date	27 / 0 5 / 2026
Hand in Date	27 / 05 / 2026
Total Marks	100
Assessment Lead Developer	Lusukama Selemani
Internal Moderator	Faith Muwishi

Student Name and Surname	Tiaan Gerber
Student Number	20240879

Instructions:

Test or Debug Source Code to Ensure Client Needs Are Met

Please read the following instructions carefully before attempting this assessment:

- Read each question carefully and consider the mark allocation before answering.
- Answer all questions in the assessment.
- Provide clear, well-structured C# code and explanations where required.
- Apply your understanding of software testing and debugging techniques when solving the given scenarios.
- Ensure that your solutions meet the client requirements described in the scenario.
- Use meaningful variable names, proper formatting, and correct programming logic.
- Test your application before submission to confirm that it runs correctly.

Submission Requirements

- For your final submission, you must include:
- A signed Declaration of Authenticity (if not already included in the assessment document provided)
- Your completed Answer Sheet (PDF document) (preferably the official template provided with this assessment)
- Your Visual Studio project files (compressed as a ZIP file)

Evidence Collection Requirements

To support your assessment submission, you must:

- Take a screenshot of the program output for each question
- Ensure each screenshot shows the full screen, including the system date and time
- Ensure screenshots clearly demonstrate correct program execution
- Do NOT submit screenshots of your source code
- Submit your complete Visual Studio project folder as a ZIP file

GitHub Submission Requirement

You are required to:

- Push your project code to GitHub
- Ensure the repository contains the correct and complete solution
- Share the GitHub repository link in your submission for verification and review
- Example: <https://github.com/your-username/project-name>

Summative Assessment Criteria:

Applied Knowledge

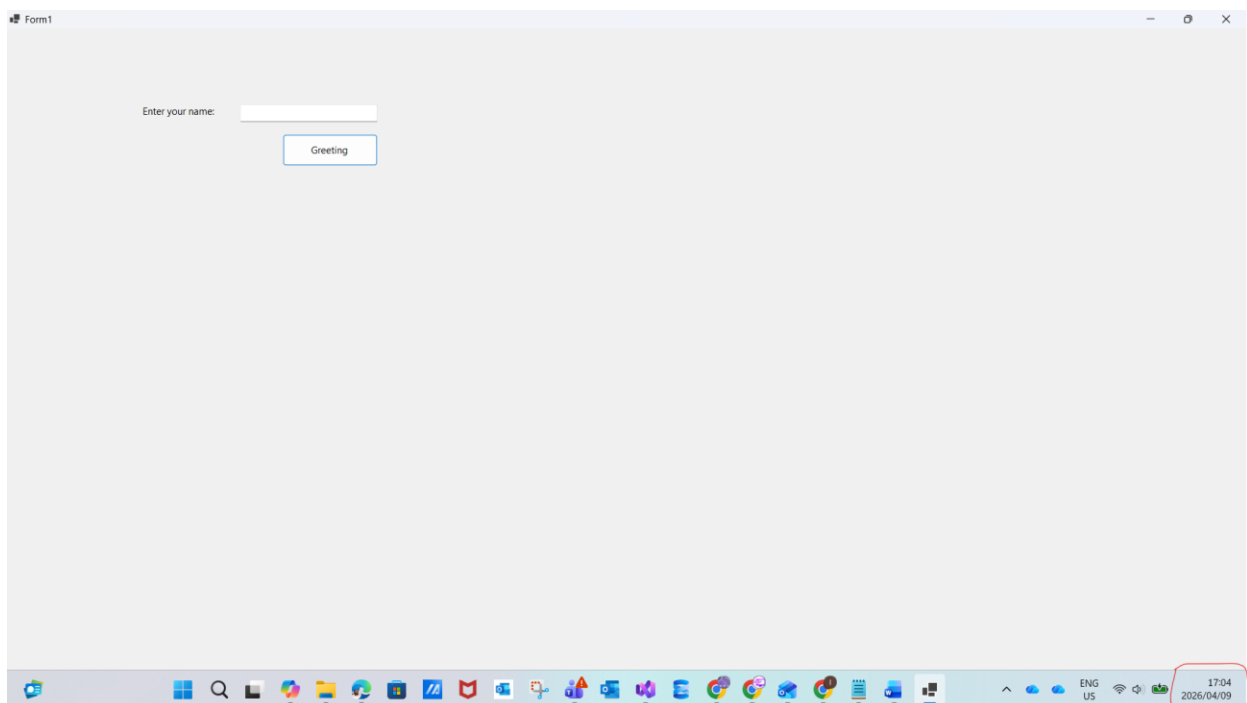
- AK0101 Knowledge of application testing techniques
- AK0102 Knowledge of test report writing

Internal Assessment Criteria

- IAC0101 functional and non-functional tests are identified
- IAC0102 The correct tests are applied
- IAC0103 A test report is produced

Example of Collecting Evidence for your submission pdf:

Question1 :



Ensure each screenshot shows the **full screen**, including the **system date and time**

SECTION A: Knowledge Questions (40 Marks)

Question 1

(30 Marks)

- 1.1 Define non-functional testing and explain what type of system qualities it evaluates. (2)

Non-functional : refers to the verification and testing of a system as like the disabeling systems and error testing the failover testing system shutdown single point of failure or chaos engineering backup and restore testing communication failure and last but not least load testing idetifying the load options and stress testing testing for system weaknesses like breaching and testing the system beyond limits

- 1.2 Explain load testing as a non-functional testing type. (1)

Non-functional testing to determine system performance under expected peak load. Load testing refers to the specified limits and the stress a system can endure under load it uses load options like static dynamic and soak load testing in load testing you must check for system metrics operation results raising system limits like checklisting hidden constraints load testing in a release cycle can be baseline CI/CD startup or shutdown and lastly the filtering of load test errors and debugging

- 1.3 Describe the difference between load testing and stress testing in non-functional test design. (2)

Load testing = testing within limits does it meet requirements?

Stress testing = testing limits how does it fail?

Load steting:

Memory leaks, race conditions, n-squared inefficiencies

Resource exhaustion handles, connections, addresses

Degraded performance under realistic peak usage

Advantages:

only way to find load-related bugs is through race conditions, memory leaks

Only way to measure system performance spec numbers must come from testing

Uncovers hard-to-find issues 1 in 1000 failures

Finds software inefficiencies n-squared algorithms

Disadvantages:

Requires dedicated, realistic, reliable test environment

Needs high-quality load generation tools tool failures drown out real issues

Stress testing:

Advantages:

ONLY way to find over-limit issues DDoS, viral spikes

Discovers real system limits not theoretical

Uncovers hard-to-find issues positive feedback loops

Prepares you for overload scenarios celebrity tweet, viral ad

Disadvantages:

Requires dedicated, high-quality tools same as load testing

Harder to debug than load testing more chaos

Less realistic scenarios 10,000x normal load
 Expensive hardware, cloud time
 Difficult to specify correct failure behavior often underspecified

1.4 Explain the difference between black-box testing and white-box testing. (3)

BLACK-BOX TESTING: Tests external behavior without knowing internal implementation

User's point of view. Most realistic Covers happy-path/main functionality

Does not cover error cases, code paths, or implementation details

Like for instance "What the system does, not how it does it."

WHITE-BOX TESTING: Tests internal implementation, code paths, and structure.

Requires coding knowledge. Also called clear-box or glass-box testing.

Complements black-box external behavior with internal coverage.

Like for instance "How it works, not just what it does."

1.5 Describe equivalence partitioning and explain how it improves functional testing efficiency (3)

Prescriptive vs Descriptive exact steps vs goals descriptive better for

Pesticide paradox same tests find fewer bugs over time

Code analysis (automatic static testing)

Static testing the manual code review dry, defensive coding

Code coverage types 9 types: function, statement, decision, condition, MC/DC, multiple condition, parameter, loop

Expected vs Unexpected errors

Fail as early as possible 4 validation layers: frontend backend storage on-read

Fail as little as possible (contain failures, no cascading - DDoS example

UX review meeting separate from spec review

* Setup vs ongoing usage onboarding disproportionately important

1.6 A Loan Eligibility Processing System accepts customer income values between:

R1 000 and R150 000 Design TWO boundary value test cases to verify correct system behaviour. (4)

So the customer input values must be between R1 000 and R150 000 if outside them it should throw an error for example

EXPECTED ERRORS

Invalid inputs, missing fields, timeouts, network issues

System should handle gracefully with user feedback

Log as WARNING not ERROR they are anticipated

UNEXPECTED ERRORS CODE BUGS:

Null pointers, divide by zero, empty lists where data expected

Should NEVER happen. Indicate code fault.

Log as ERROR needs developer fix

"Expected errors should not become unexpected. Test to ensure they don't escalate."

Excessive logging on error can flood discs

Infinite retry loops spams other systems

USER FEEDBACK REQUIREMENTS:

What went wrong? specific field/operation

Is it user's fault? fixable by them

Is it temporary? worth retrying?

Is it permanent? don't retry

1.7 Define error-case testing.

(1)

ERROR CASE TESTING: Tests how system behaves when things go wrong.

Expected failures invalid inputs, missing data, timeouts

not unexpected bugs null pointers, divide by zero those are code bugs

The advantages includes Product owners often overlook error cases your chance to add value

Developers may not have considered them bugs waiting to be found

And the disadvantages includes Lower priority than happy path rare conditions

Hard to generate specific error cases need tools/mocks

1.8 Explain why testing unexpected user inputs is important when validating software reliability. (2)

It is important to test for these things because they test for Outdated software/libraries known vulnerabilities

Outdated hash functions can be broken

Unencrypted connections eavesdropping

Common web vulnerabilities

Security testing: Extension of functional testing focused on CIA triad:

confidentiality: Data only to authorized owners

integrity: Data accurate and unaltered

availability: Data accessible when needed

Best way to discover security vulnerabilities

Many automated tools available don't reinvent

Easy to automate scanning

Threats constantly evolve

1.9 Provide TWO examples of error-case tests suitable for a login system. (2)

Frontend best:

Web page, app, API entry point

Instant user feedback No network round-trip

Example: Email format check in browser no @ symbol reject immediately

backend:

Server, cloud infrastructure

Returns helpful error with details

Logs what was received and why rejected

Username tests:

Don't indicate if username exists response time same, message generic

Signup: balance security vs UX tell user if taken? depends on service

Handle email sub-addressing (joe+anything@) and dots Gmail treats as same

Password tests:

Complexity requirements numbers, special chars, length

Rate limits on guesses block automated attacks

Reset requires CURRENT password unless forgotten email reset link

Reset invalidates ALL existing sessions

Passwords must be hashed never plain text, never email password back

Work with password managers

1.10 Define a test case in software testing.

(2)

Prerequisites Versions, services, config needed before test suite

setup Initial state user, data, cookies, registry

procedure Steps to execute

result Expected outcome what to check

perspective Low-level: "Upload /fixtures/image1.jpeg"

+ Quick to execute, junior staff can run, easily automatable

Limits creativity, reduces coverage, dull for testers

descriptive High-level: "Upload a .jpeg image between 100KB-5MB"

+ Varied testing, avoids pesticide paradox, uses tester experience

Less reproducible, needs experienced staff, may be done wrong

Verdicts for manual tests : Descriptive is better more coverage, finds more bugs

Verdict for automated tests: Must be prescriptive but rotate test data

1.11 List THREE components typically included in a structured test case.

(3)

A structure test case often refers to the testing pyramid : System testing Integration testing and unit testing

Unit testing Simple, reliable, fast, easy debug Refactoring breaks tests, unrealistic

Integration testing Decouples teams, matches modules Stubs take time, risk false positives

System testing Most realistic, finds all bug types Slow, hard debug, all components needed

prerequisites Versions, services, config needed before test suite

setup Initial state user, data, cookies, registry

procedure Steps to execute

result Expected outcome what to check

1.12A testing session executed:

Test Cases	Passed	Failed
100	80	20

Prepare a structured test summary section for a system testing report including:

Test results overview

Recommendation to stakeholders (5)

100 test cases in total of which 80 passed and 20 failed

What the test explorer shows for these test cases:

All test methods in the solution (green = 80 that passed, red =for 20 fail, blue = not run at all)

Run all Run passed Run failed Run not run buttons

Duration of each test

Output/error messages when test fails

Test [name] failed because reason

Expected: [value]

Actual: [value]

Fix applied: [code change]

After fix, test now passes (see screenshot)

Test Case ID	Description	Expected Result	Actual Result	Pass/Fail
Tc001 validage	Age =25	accepted	pass	pass
Tc002 negativeage	Age= -5	Reject with error	rejected	pass
Tc003 zeroage	Age=0	Reject if min age1	Accepted bug	Fail
Tc004 nullname	Name = null	Reject with error	Exception crash	fail

For each failed test (in your report):

TC003 failed because the system accepted age 0 when minimum age is 1.

Root cause: Validation condition was age < 0 instead of age <= 0.

Fix: Changed if (age < 0) to if (age <= 0).

After fix, test passes see screenshot."

Total _____ / 30

True / False Questions

[10]

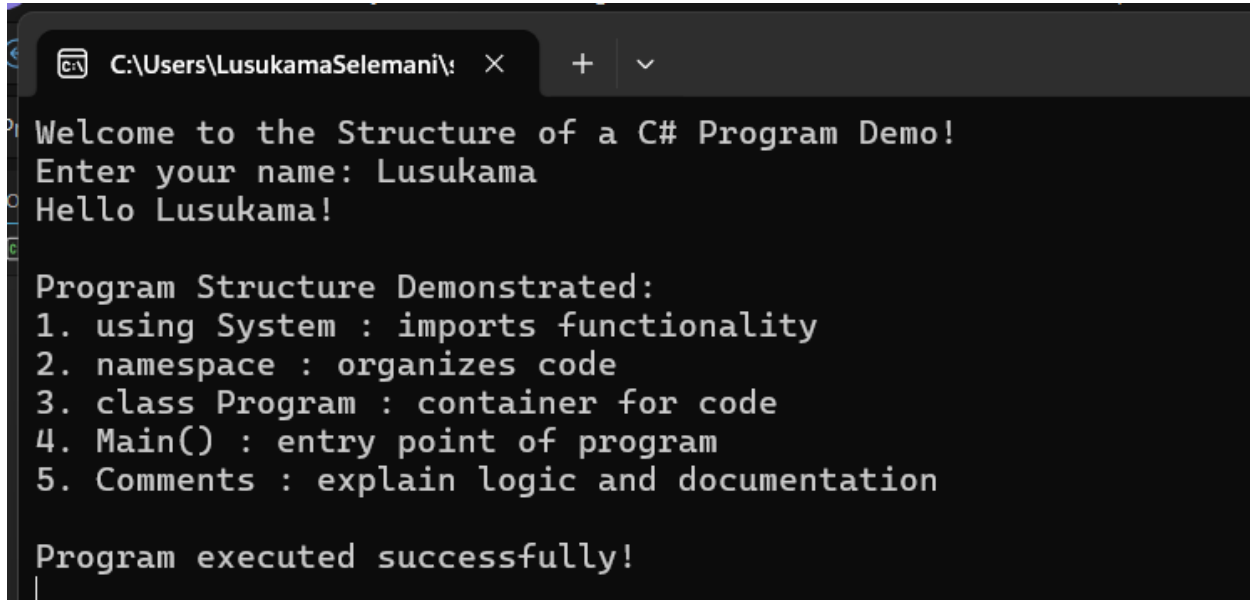
True / False Question	Answer
Exploratory testing allows testers to learn, design, and execute tests at the same time.	True
A good feature specification should be vague to allow flexibility during testing.	False
Specification review meetings help identify missing or incorrect requirements before testing begins.	True
Unit testing is typically performed after system testing.	False
Black-box testing focuses on testing system behavior without examining internal code structure.	True
White-box testing examines internal program logic and code structure.	True
Error-case testing helps ensure the system handles unexpected inputs correctly.	True
UX testing only evaluates system performance and ignores usability.	False
Security testing includes checking authentication and authorization mechanisms.	True
Maintainability testing helps determine how easily a system can be monitored and updated.	True

Total: ____ / 10

SECTION B: Short Practical Questions (10 Marks)

Question 1

1.1 Create a C# Console Application that the display the below output: (5)



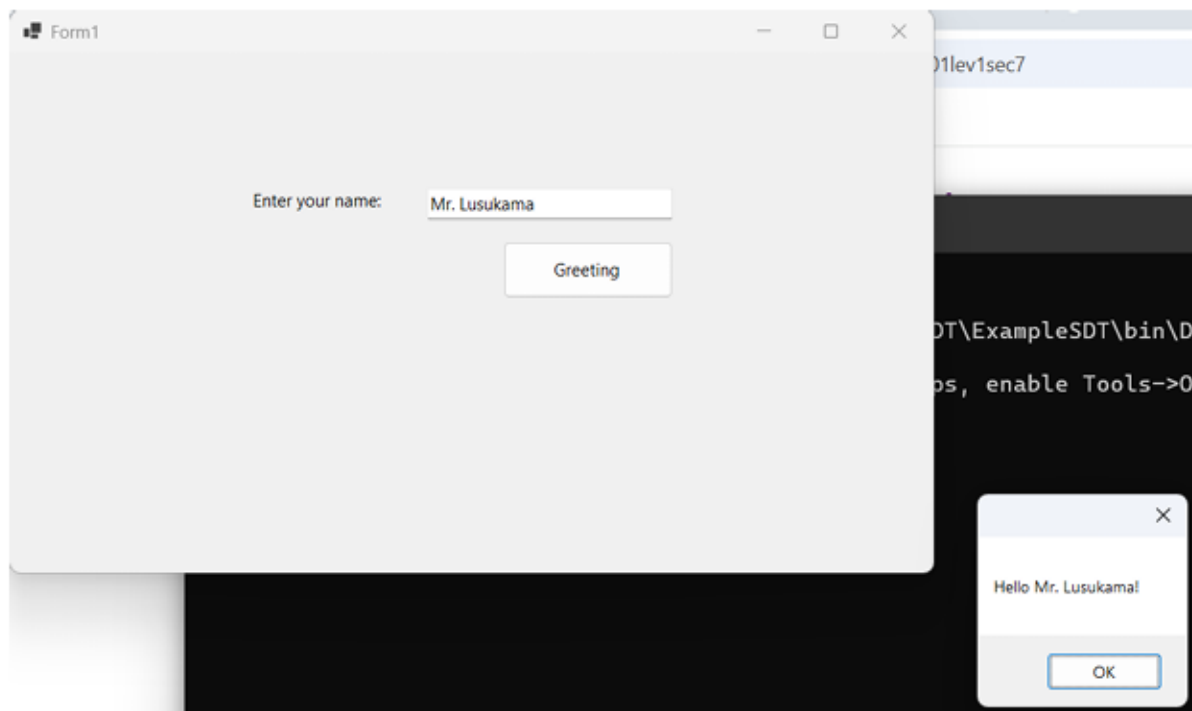
```

C:\Users\LusukamaSelemani\>
Welcome to the Structure of a C# Program Demo!
Enter your name: Lusukama
Hello Lusukama!

Program Structure Demonstrated:
1. using System : imports functionality
2. namespace : organizes code
3. class Program : container for code
4. Main() : entry point of program
5. Comments : explain logic and documentation

Program executed successfully!
  
```

1.2 Develop a Windows Forms Application that allows the user to enter their name and display a greeting message. (5)



SECTION C: Scenario-Based Practical [50 Marks]

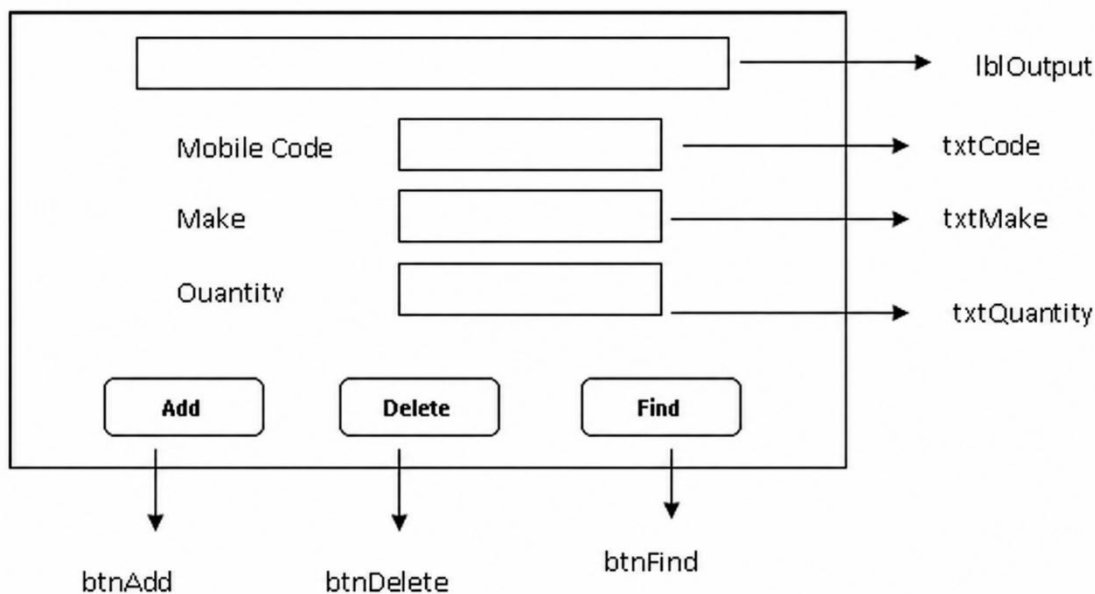
Question 1

You are working as a Junior Software Tester in a retail inventory company. The development team created a Mobile Stock Capture Windows Forms application, but the system contains interface naming issues, missing validations, and incorrect button behavior.

You are required to design, test, debug, and improve the Windows Forms application shown in the diagram.

SECTION C.1 — User Interface Design (10 Marks)

- 1.1 Create and design the following user interface using appropriate C# Windows Forms App controls and components. (10)



SECTION C.2 — Button Event Logic Implementation (20 Marks)

- 1.2 Write the events for each button

1.2.1 When the **btnAdd** button is clicked, the program should insert a record into the **tblMobilePhones** with the **MobileCode**, **Make** and **Quantity** from the corresponding data from the form and display the message "Record Added" in the **lblOutput**. (8)

1.2.2 When the **btnDelete** button is clicked, the program should delete the record whose **MobileCode** is typed in the **txtCode** text field and display the message "Record Found" in the **lblOutput**; or "Record NOT Found" if the record is not found. (7)

1.2.3 When the **btnFind** is clicked, the program should find the record whose **MobileCode** is typed in the **txtCode** text field and display the message "Record Deleted" in the **lblOutput**. (5)

Note: use functional and non-functional tests for this question

SECTION C.3 : Software Testing Report (15 Marks)

1.3 Produce a Software Testing Report

Using the results from Question 1, compile a professional software testing report for the Mobile Stock Capture Application.

Your report must include the following sections:

1.3. 1 Application Description (3 Marks)

Provide a brief description of:

- Purpose of the application
- Main system functions
- User inputs captured

1.3.2 Test Cases Table (5 Marks)

Create a table similar to the example below:

Test Case	Input	Expected Result	Actual Result	Status

Include at least five test cases.

1.3.3 Identified Defects (4 Marks)

List any problems found during testing:

Click or tap here to enter text.

1.3.4. Recommendations for Improvement (3 Marks)

Click or tap here to enter text.

SECTION C.4 — GitHub Version Control Evidence (5 Marks)

1.3.5 Use of GitHub

To obtain the full 5 marks, learners must demonstrate correct use of GitHub version control by completing the following tasks:

- Create and work within a separate branch (not directly on the main branch)
- Make meaningful and professional commit messages that clearly describe the changes made
- Successfully merge the branch into the main (origin/main) branch after completing development
- Ensure the repository contains the complete working solution
- Submit a valid and accessible GitHub repository link

End Exam

SECTION A — Knowledge Questions (40 Marks)

Purpose of Section A

Assess theoretical understanding of testing techniques, validation strategies, and structured test documentation required before practical debugging begins.

QCTO Mapping

Question Area	Marks	QCTO Criterion	Evidence Demonstrated
Non-functional testing definition	2	AK0101	Knowledge of testing concepts
Load testing explanation	1	AK0101	Understanding performance testing
Load vs Stress testing comparison	2	AK0101	Distinguishing test strategies
Black-box vs White-box testing	3	AK0101	Test method classification
Equivalence partitioning	3	AK0101	Functional test design
Boundary value test cases	4	AK0101	Boundary testing application
Error-case testing definition	1	AK0101	Exception scenario awareness
Unexpected input validation importance	2	AK0101	Reliability assurance concepts
Login error-case examples	2	AK0101	Functional validation thinking
Definition of test case	2	AK0102	Test documentation knowledge
Components of test case	3	AK0102	Structured testing documentation
Test summary & stakeholder recommendation	5	AK0102	Test reporting interpretation

Section A Mapping Summary

Criterion	Marks
AK0101 Knowledge of testing techniques	25
AK0102 Knowledge of test report writing	15

Total Section A = 40 Marks

SECTION B — Short Practical Questions (10 Marks)

Purpose of Section B

Assess ability to implement and validate basic working software functionality before applying structured testing.

QCTO Mapping Table

Task	Marks	QCTO Criterion	Evidence Demonstrated
Console output application	5	IAC0102	Correct functional output validation
Windows Forms greeting application	5	IAC0102	UI interaction testing

Section B Mapping Summary

Criterion	Marks
IAC0102 Correct tests applied	10

Total Section B = 10 Marks

SECTION C — Scenario-Based Practical (50 Marks)

Purpose of Section C

Assess full PM-04 competency:

Test or debug source code to ensure client needs are met

Learners must design, debug, validate, test, and document results.

SECTION C.1 — User Interface Design (10 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Correct UI controls selected	2	IAC0101	Functional test readiness
Correct control naming	2	IAC0101	Interface clarity
Layout alignment with specification	3	IAC0101	UI correctness verification
Proper component usage	3	IAC0101	Interface usability validation

Mapping Summary

Criterion	Marks
IAC0101 Functional & non-functional tests identified	10

SECTION C.2 — Button Event Logic Implementation (20 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Add button functionality	8	IAC0102	Record insertion testing
Delete button functionality	7	IAC0102	Conditional logic validation
Find button functionality	5	IAC0102	Search behaviour verification

Mapping Summary

Criterion	Marks
IAC0102 Correct tests applied	20

SECTION C.3 — Software Testing Report (15 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Application description	3	AK0102	Documentation understanding
Test cases table	5	IAC0102	Test execution structure
Identified defects	4	IAC0101	Fault detection ability

Recommendations	3	IAC0103	Improvement reporting
-----------------	---	---------	-----------------------

SECTION C.4 — GitHub Version Control Evidence (5 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Branch created and used	1	IAC0102	Version control workflow
Professional commit messages	1	IAC0102	Change tracking clarity
Multiple commits recorded	1	IAC0102	Development progression evidence
Branch merged into main	1	IAC0102	Integration workflow
Repository complete & accessible	1	IAC0103	Submission verification

SECTION C Mapping Summary

Criterion	Marks
IAC0101 Tests identified	14
IAC0102 Tests applied	26
IAC0103 Test report produced	10

Total Section C = 50 Marks

FINAL OVERALL PM-04 ASSESSMENT SUMMARY

QCTO Criterion	Description	Marks
AK0101	Knowledge of testing techniques	25
AK0102	Knowledge of test reporting	18
IAC0101	Functional & non-functional tests identified	14
IAC0102	Correct tests applied	36
IAC0103	Test report produced	7

TOTAL = 100 MARKS